Cloud Computing

The Ultimate Self-Study Resource - No Courses Needed!

AWS | Azure | GCP | Docker | Kubernetes | Serverless | DevOps | Interview Prep

Creator & Author

Mohammad Faiz

Year 2026

Table of Contents

Part 1: Cloud Computing Fundamentals 4

What is Cloud Computing? .....

Essential Cloud Characteristics .....

Cloud Service Models (IaaS, PaaS, SaaS) .....

Cloud Deployment Models .....

Cloud Economics and Pricing .....

Part 2: Amazon Web Services (AWS) 14

AWS Compute Services (EC2, Lambda, ECS) .....

AWS Storage Services (S3, EBS, EFS) .....

AWS Database Services (RDS, DynamoDB) .....

AWS Networking (VPC, Route 53, CloudFront) .....

AWS Security (IAM, KMS, CloudTrail) .....

Part 3: Microsoft Azure 27

Azure Compute and Storage .....

Azure Databases and Networking .....

Azure Identity and Security .....

Part 4: Google Cloud Platform (GCP) 33

GCP Compute and Storage .....

GCP Data Analytics and ML .....

Part 5: Containers and Docker 37

Docker Fundamentals .....

Dockerfile and Multi-Stage Builds .....

Docker Compose and Networking .....

Part 6: Kubernetes 42

Kubernetes Architecture .....

Kubernetes Objects (Pods, Deployments, Services) .....

Helm and kubectl Commands .....

Part 7: Serverless Computing **48**

Function as a Service (FaaS) .....

Cold Starts and Best Practices .....

Part 8: Infrastructure as Code **52**

Terraform Deep Dive .....

CloudFormation and Pulumi .....

Part 9: CI/CD and DevOps **57**

CI/CD Pipelines and GitOps .....

Deployment Strategies .....

Observability and Monitoring .....

Part 10: Cloud Architecture Patterns **63**

Well-Architected Framework .....

Microservices and 12-Factor App .....

High Availability and Disaster Recovery .....

Cost Optimization (FinOps) .....

Part 11: Interview Preparation **67**

Common Interview Questions .....

Scenario-Based Questions .....

Career Path and Certifications .....

Hands-On Labs and Projects .....

Part 1: Cloud Computing Fundamentals

What is Cloud Computing?

Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (networks, servers, storage, applications, services) that can be rapidly provisioned and released with minimal management effort.

Before Cloud vs With Cloud

```
BEFORE CLOUD (Traditional IT):
1. Buy servers (hardware procurement: 4-8 weeks)
2. Rack in data center (setup: 1-2 weeks)
3. Install operating system (configuration: 2-3 days)
4. Configure network (firewalls, switches: 1 week)
5. Deploy application (installation: 1-2 days)
6. Set up monitoring and backups
7. Scale by buying MORE servers (repeat steps 1-6)

Timeline: Weeks to months
Cost: $100,000+ upfront capital expenditure
Commitment: 3-5 year hardware lifecycle


WITH CLOUD:
1. Log into cloud console
2. Click 'Launch Instance' or run CLI command
3. Server ready in 60 seconds
4. Deploy application immediately
5. Scale automatically with a few clicks

Timeline: Minutes
Cost: Pay-as-you-go, no upfront cost
Commitment: Hourly or even per-second billing
```

TIP: Real-World Analogy

Think of cloud computing like electricity. In the past, businesses had to build their own power plants. Now, you simply plug into the wall and pay for what you use. The power company handles generation, maintenance, and scaling. Cloud computing works the same way - instead of building and maintaining your own IT infrastructure, you use resources from cloud providers and pay only for what you consume.

Why Cloud Computing?

- Cost Savings: No upfront hardware costs, pay only for what you use (OpEx vs CapEx)
- Scalability: Instantly scale resources up or down based on demand (elasticity)
- Reliability: Built-in redundancy and automatic failover across data centers
- Global Reach: Deploy applications worldwide in minutes with edge locations
- Security: Enterprise-grade security managed by experts (shared responsibility)
- Speed: Provision resources in minutes instead of weeks or months
- Innovation: Access cutting-edge technologies like AI/ML without building expertise

Essential Cloud Characteristics (NIST)

The National Institute of Standards and Technology (NIST) defines five essential characteristics of cloud computing:

Characteristic	Description	Real Example
On-Demand Self-Service	Provision resources automatically without human intervention	Launch an EC2 instance from AWS Console without calling anyone
Broad Network Access	Access via standard internet protocols from anywhere	SSH into servers from your laptop, tablet, or phone
Resource Pooling	Provider's resources shared across customers (multi-tenancy)	AWS runs millions of VMs on shared hardware securely
Rapid Elasticity	Scale up/down automatically based on demand	Auto-scaling group adds 10 servers during Black Friday traffic spike
Measured Service	Pay for what you use, monitored and billed	Per-second billing for compute, per-GB for storage

Understanding Multi-Tenancy

Multi-tenancy is the foundation of cloud economics. Multiple customers (tenants) share the same physical infrastructure while remaining logically isolated.

- Think of it like an apartment building - many tenants share the same structure, but each has their own secure, private space
- Hypervisors (like VMware, Xen, KVM) create virtual machines that share physical CPU, memory, and storage
- Each tenant's data and workloads are isolated and secure from other tenants
- This sharing enables lower costs for everyone while maintaining security

Cloud Service Models

Cloud services are categorized into models based on who manages what. Understanding these models is crucial for making the right architectural decisions.

Infrastructure as a Service (IaaS)

IaaS provides the fundamental building blocks of computing - virtual machines, storage, and networks. You have the most control but also the most responsibility.

- What You Manage: Operating System, applications, data, runtime, middleware, networking configuration
- What Provider Manages: Physical servers, data center facilities, virtualization layer, network infrastructure
- Best For: Organizations wanting maximum control, migrating existing applications (lift-and-shift)
- Examples: AWS EC2, Azure Virtual Machines, Google Compute Engine

IaaS Real-World Example

A retail company wants to move their e-commerce website to the cloud.

Using IaaS (AWS EC2), they:
- Create virtual machines with their preferred OS (Ubuntu Linux)
- Install and configure their web server (Apache/Nginx)
- Deploy their PHP/Java application code
- Set up their MySQL database on an EC2 instance
- Configure security groups (firewall rules)
- Manage OS patches and updates themselves

They control everything but don't need to buy physical servers.

Platform as a Service (PaaS)

PaaS provides a platform for developers to build, run, and manage applications without worrying about the underlying infrastructure.

- What You Manage: Applications and data only
- What Provider Manages: Runtime, middleware, operating system, virtualization, servers, storage, networking
- Best For: Developers focusing on code, rapid application development, startups
- Examples: AWS Elastic Beanstalk, Azure App Service, Google App Engine, Heroku

PaaS Real-World Example

A startup is building a new mobile app backend.

Using PaaS (Heroku), they:
- Write their application code in Node.js
- Push code to Heroku using Git: git push heroku main
- Heroku automatically handles deployment, scaling, and infrastructure
- They focus entirely on building features, not managing servers
- Database is provisioned with a simple command: heroku addons:create heroku-postgresql

No server management needed - just code and deploy.

Software as a Service (SaaS)

SaaS delivers fully functional applications over the internet. Users simply access the software through a web browser or mobile app.

- What You Manage: Nothing - just use the software
- What Provider Manages: Everything - application, data, runtime, middleware, OS, infrastructure
- Best For: End users and businesses wanting ready-to-use solutions
- Examples: Gmail, Salesforce, Microsoft 365, Zoom, Dropbox, Slack

SaaS Real-World Example

A small business needs email and document collaboration.

Using SaaS (Google Workspace), they:
- Sign up for Google Workspace accounts online
- Start using Gmail, Google Docs, Sheets, and Drive immediately
- No installation, no servers, no maintenance required

- Pay per user per month ($6-18/user depending on plan)
- Automatic updates and new features

Everything is managed by Google - they just use the tools.

Newer Service Models

Model	Description	Example Services
FaaS (Functions as a Service)	Run individual functions without managing servers	AWS Lambda, Azure Functions, Google Cloud Functions
CaaS (Containers as a Service)	Run containers without managing K8s clusters	AWS Fargate, Azure Container Instances, Google Cloud Run
DBaaS (Database as a Service)	Fully managed databases	AWS RDS, Azure SQL, Google Cloud SQL
BaaS (Backend as a Service)	Complete backend for mobile/web apps	Firebase, Supabase, AWS Amplify

Service Model Comparison

Aspect	IaaS	PaaS	SaaS
Control Level	High (full OS control)	Medium (app-focused)	Low (just use software)
Management Overhead	High (you manage OS)	Medium (you manage app)	None (provider manages all)
Flexibility	Maximum (any software)	Good (limited by platform)	Limited (what app offers)
Time to Deploy	Hours/Days	Minutes/Hours	Immediate
Technical Skill Required	High (sysadmin skills)	Medium (developer skills)	Low (end user)
Customization	Complete	Platform-dependent	Configuration only

Cloud Deployment Models

Organizations can deploy cloud resources in different ways depending on their security, compliance, and operational requirements.

Public Cloud

Public cloud resources are owned and operated by third-party cloud providers and delivered over the internet. The infrastructure is shared among multiple organizations (tenants).

- Advantages: No capital expenditure, unlimited scalability, pay-per-use pricing, global reach
- Disadvantages: Less control over infrastructure, potential security concerns for sensitive data, dependency on internet connectivity
- Best For: Startups, variable workloads, public-facing applications, development/testing
- Major Providers: AWS (33% market share), Azure (23%), Google Cloud (11%), IBM Cloud, Oracle Cloud

Private Cloud

Private cloud resources are used exclusively by a single organization. The infrastructure can be on-premises or hosted by a third party.

- Advantages: Maximum control, enhanced security, compliance-friendly (HIPAA, PCI-DSS), dedicated resources
- Disadvantages: Higher costs (CapEx), requires IT expertise, limited scalability, longer provisioning times
- Best For: Highly regulated industries (healthcare, finance), sensitive data, legacy applications
- Technologies: VMware vSphere, OpenStack, Microsoft Azure Stack, AWS Outposts

Hybrid Cloud

Hybrid cloud combines public and private clouds, allowing data and applications to move between them as needed.

- Advantages: Flexibility, cost optimization, disaster recovery, compliance (keep sensitive data private)
- Disadvantages: Complexity, integration challenges, security management across environments
- Best For: Organizations transitioning to cloud, burst capacity needs, regulatory compliance
- Use Case: Keep sensitive customer data in private cloud, use public cloud for web tier and

analytics

Multi-Cloud

Multi-cloud uses services from multiple cloud providers to avoid vendor lock-in and leverage best-of-breed services.

- Advantages: Avoid vendor lock-in, best service selection, redundancy, negotiating leverage
- Disadvantages: Increased complexity, skills requirements across platforms, integration challenges
- Best For: Large enterprises, avoiding single points of failure, leveraging provider strengths
- Example: AWS for compute (breadth), GCP for ML (BigQuery, Vertex AI), Azure for Office 365 integration

Deployment Model Decision Example

A healthcare provider choosing their cloud strategy:

```
PRIVATE CLOUD (On-Premises):
- Store patient records (HIPAA compliance requirement)
- Run legacy hospital management software

PUBLIC CLOUD (AWS):
- Run patient portal website
- Process medical imaging with AI/ML
- Store anonymized research data

HYBRID CONNECTION:
- AWS Direct Connect for secure, private connection
- VPN for encrypted data transfer
- API Gateway for secure access to patient data
```

This gives them compliance for sensitive data while leveraging public cloud benefits.

Cloud Economics and Pricing

Understanding cloud pricing is essential for cost management and making informed architectural decisions.

Cloud Pricing Models

Model	Description	Best For	Savings
On-Demand	Pay per hour/second, no commitment	Variable workloads, testing, short-term	0% (baseline)
Reserved Instances	Commit for 1-3 years upfront	Stable, predictable workloads	30-72% savings
Spot/Preemptible	Use spare capacity (can be terminated)	Fault-tolerant workloads, batch jobs	60-90% savings
Savings Plans	Commit to usage level ($/hour) for 1-3 years	Flexible commitment across instance types	Up to 72% savings

Total Cost of Ownership (TCO) Comparison

```
ON-PREMISES (Traditional Data Center):
Server hardware:        $50,000 (upfront)
Network equipment:      $10,000 (upfront)
Power & cooling:        $12,000/year
Data center space:
$24,000/year IT staff (2 FTE):
                        $200,000/year
Software licenses:
                        $15,000/year
Maintenance:            $8,000/year
-----------------------------------------------------------------------------
Total Year 1:           ~$319,000
Total Year 2-3:         ~$259,000/year

CLOUD (IaaS):
EC2 instances (3x t3.large):
$15,000/year Data transfer:
                        $500/year
Storage (1TB SSD):      $1,200/year
Load balancer:          $600/year
Backup storage:         $400/year
-----------------------------------------------------------------------------
Total per year:         ~$17,700

SAVINGS: ~94% in Year 1, ~93% ongoing
PLUS: No upfront capital expenditure!
```

Cost Optimization Strategies

- Right-Sizing: Match instance type to actual workload needs (don't oversize)
- Auto-Scaling: Scale down during low traffic periods (nights, weekends)
- Reserved Instances: Use for predictable, steady-state workloads
- Spot Instances: Use for fault-tolerant workloads (batch processing, ML training)
- Storage Tiering: Move infrequently-accessed data to cheaper storage classes
- Delete Unused Resources: Terminate idle instances, unattached volumes, old snapshots
- Tagging Strategy: Tag all resources (Environment, Team, Project) for cost allocation

TIP: FinOps Best Practice

Cloud cost optimization is not a one-time activity - it's an ongoing discipline. Set up budgets and alerts, review costs monthly, and make cost awareness part of your engineering culture. Tools like AWS Cost Explorer, Azure Cost Management, and third-party solutions like CloudHealth can help visualize and optimize spending.

Cloud Regions and Availability

Cloud providers operate data centers worldwide, organized into regions and availability zones.

- Region: A geographic area containing multiple data centers (e.g., us-east-1, eu-west-1)
- Availability Zone (AZ): One or more isolated data centers within a region with independent power, cooling, and networking
- Each region has 2-6 AZs, typically tens of miles apart for disaster isolation
- Edge Locations: CDN points of presence for caching content close to users (400+ worldwide)

Service	Availability SLA	What It Means
AWS EC2 (Single AZ)	99.5%	~43 hours downtime/year
AWS EC2 (Multi-AZ)	99.99%	~52 minutes downtime/year
AWS S3 Standard	99.99% availability	~52 minutes downtime/year
AWS S3 Durability	99.999999999% (11 nines)	Virtually zero data loss
Azure VMs (with AZs)	99.99%	~52 minutes downtime/year
GCP Compute Engine	99.99%	~52 minutes downtime/year

WARNING: Common Pitfall

Many organizations start with a single AZ deployment to save costs, then experience outages when that

AZ has issues. For production workloads, always deploy across multiple AZs. The small additional cost is worth the significantly higher availability.

Part 2: Amazon Web Services (AWS)

AWS is the world's largest cloud provider with 33% market share and 200+ fully featured services. Launched in 2006, AWS pioneered the cloud computing industry and remains the most feature-rich platform.

AWS Compute Services

EC2 (Elastic Compute Cloud)

EC2 provides resizable compute capacity in the cloud. It is like having a virtual computer that you can configure and control completely.

```
# Launch an EC2 instance via AWS CLI
aws ec2 run-instances \
  --image-id ami-0abcdef1234567890 \
  --instance-type t3.medium \
  --key-name my-key \
  --security-group-ids sg-0123456789abcdef0 \
  --subnet-id subnet-0bb1c79de3EXAMPLE \
  --count 1 \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=WebServer}]'
```

EC2 Instance Families

Family	Optimized For	Example Use Case	Price Range
T3/T3a	General purpose (burstable)	Dev/test, small apps, low-traffic websites	$0.004-0.16/hour
M6i/M7g	General purpose (steady)	Databases, app servers, enterprise apps	$0.10-0.40/hour
C6i/C7g	Compute-intensive	Batch processing, gaming servers, ML inference	$0.085-0.34/hour
R6i/R7g	Memory-intensive	In-memory caches, real-time analytics, SAP	$0.13-0.50/hour

Family	Optimized For	Example Use Case	Price Range
P4/P5	GPU (accelerated)	ML training, video rendering, scientific computing	$3.06-32.77/hour
I3/I4i	Storage-intensive	Data warehouses, Elasticsearch, NoSQL	$0.16-0.62/hour
Graviton (g suffix)	ARM-based processors	40% better price-performance than Intel	20-40% cheaper

EC2 Instance Selection Guide

Small Web Server (low traffic):
- t3.micro (2 vCPU, 1 GB RAM) - $0.0104/hour
- Free tier eligible (750 hours/month)

Medium Application Server:
- m6i.large (2 vCPU, 8 GB RAM) - $0.096/hour
- Good for most web applications

High-Performance Database:
- r6i.xlarge (4 vCPU, 32 GB RAM) - $0.268/hour
- Memory-optimized for databases

Machine Learning Training:
- p3.2xlarge (8 vCPU, 61 GB RAM, 1x V100 GPU) - $3.06/hour
- Use Spot instances for 70% savings on training

EC2 Pricing Models Deep Dive

Pricing Model	How It Works	Best For	Potential Savings
On-Demand	Pay per hour/second, no commitment	Variable workloads, testing, dev	Baseline (0%)
Reserved (1-year)	Commit to 1 year, pay upfront or monthly	Steady-state production workloads	30-40%
Reserved (3-year)	Commit to 3 years, all upfront	Long-term infrastructure	50-60%

Pricing Model	How It Works	Best For	Potential Savings
Spot	Bid on spare capacity, can be interrupted	Fault-tolerant, flexible workloads	60-90%
Savings Plans	Commit to $/hour usage for 1-3 years	Flexible across instance types	Up to 72%

AWS Lambda (Serverless)

Lambda lets you run code without provisioning or managing servers. You pay only for the compute time you consume.

```
# Python Lambda function handler
import json

def lambda_handler(event, context):
    name = event.get('name', 'World')
    return {
        'statusCode': 200,
        'body': json.dumps(f'Hello, {name}!')
    }
```

Feature	Limit	Notes
Max execution time	15 minutes	For longer tasks, use ECS/Fargate
Memory	128 MB to 10,240 MB	Allocated in 1 MB increments
Deployment package	50 MB zipped, 250 MB unzipped	Use Lambda layers for shared code
Concurrent executions	1,000 (default)	Can be increased via support request
Environment variables	4 KB total	Use AWS Secrets Manager for secrets
Pricing	$0.20 per 1M requests + $0.0000166667 per GB-second	1M free requests/month

Lambda Trigger Sources

- API Gateway: HTTP requests to Lambda (REST APIs)
- S3 Events: File uploads trigger processing functions
- SQS: Process messages from queues asynchronously
- DynamoDB Streams: React to database changes in real-time
- CloudWatch Events: Scheduled cron-like jobs
- EventBridge: Event-driven architecture with custom events
- Cognito: Custom authentication flows

ECS and EKS (Containers)

- ECS (Elastic Container Service): AWS-native container orchestration, simpler than Kubernetes
- EKS (Elastic Kubernetes Service): Managed Kubernetes on AWS, most popular container orchestrator
- Fargate: Serverless containers - run containers without managing EC2 instances

ECS vs EKS Decision Guide

```
Choose ECS when:
- You want AWS-native integration
- Simpler learning curve is important
- Deep Kubernetes expertise not available
- Cost optimization is priority

Choose EKS when:
- You need Kubernetes portability (multi-cloud)
- Rich ecosystem of K8s tools needed
- Team has Kubernetes expertise
- Complex workloads requiring advanced features
```

AWS Storage Services

S3 (Simple Storage Service)

S3 is object storage built to store and retrieve any amount of data from anywhere. It is designed for 99.999999999% (11 nines) durability.

```
# S3 CLI
Commands # Create
a bucket
```

```
# Upload a file
aws s3 cp backup.tar.gz s3://my-app-data-2024/backups/

# Sync a directory
aws s3 sync ./build/ s3://my-app-data-2024/static/ --delete

# List bucket contents
aws s3 ls s3://my-app-data-2024/ --recursive --human-readable --summarize
```

S3 Storage Classes

Storage Class	Access Pattern	Cost	Retrieval	Best For
S3 Standard	Frequent access	$$$	Instant	Active websites, content distribution
S3 Intelligent-Tiering	Unknown/ changing	Auto-optimizes	Instant	Data with unpredictable access
S3 Standard-IA	Infrequent (monthly)	$$	Instant (retrieval fee)	Backups, older data
S3 One Zone-IA	Infrequent, non-critical	$	Instant (retrieval fee)	Secondary backups
S3 Glacier Instant	Archive, need instant	$	Instant	Medical images, news archives
S3 Glacier Flexible	Archive (quarterly)	¢	1-12 hours	Compliance archives
S3 Glacier Deep Archive	Long-term (yearly)	¢¢	12-48 hours	Regulatory compliance

S3 Lifecycle Policy Example

Automatically transition objects to cheaper storage:

Day 0-30: S3 Standard (active use)
Day 31-90: S3 Standard-IA (infrequent access)
Day 91-365: S3 Glacier Flexible (archival)

Day 366+: S3 Glacier Deep Archive (long-term)

This can reduce storage costs by 80-95% for older data.

EBS (Elastic Block Store)

EBS provides persistent block storage volumes for EC2 instances.

Volume Type	Use Case	Performance	Cost
gp3	General purpose SSD (recommended)	3,000-16,000 IOPS	$0.08/GB-month
gp2	General purpose SSD (legacy)	3 IOPS/GB burst to 3,000	$0.10/GB-month
io2	High-performance databases	64,000-256,000 IOPS	$0.125/GB + $0.065/IOPS
st1	Throughput-intensive HDD	500 MB/s per TB	$0.045/GB-month
sc1	Cold HDD (lowest cost)	250 MB/s per TB	$0.025/GB-month

EFS (Elastic File System)

EFS is a managed NFS service that can be mounted by multiple EC2 instances simultaneously.

- Use Case: Shared content storage, CMS, WordPress, home directories
- Scales automatically from gigabytes to petabytes
- Supports thousands of concurrent connections
- Storage classes: Standard (frequent) and IA (infrequent)

AWS Database Services

RDS (Relational Database Service)

RDS manages relational databases, handling provisioning, patching, backup, recovery, and scaling.

Database Engine	Version	Best For
MySQL	5.7, 8.0	Web applications, WordPress,

Database Engine	Version	Best For
		LAMP stack
PostgreSQL	12-16	Enterprise apps, geospatial, JSON support
MariaDB	10.6+	MySQL-compatible, open-source
Oracle	19c, 21c	Enterprise legacy applications
SQL Server	2019, 2022	Windows-based applications

RDS Multi-AZ Deployment

Multi-AZ provides high availability by automatically provisioning a standby replica in a different Availability Zone.

- Synchronous replication: Data written to primary is 同步 replicated to standby
- Automatic failover: If primary fails, RDS automatically promotes standby (60-120 seconds)
- Zero data loss: Synchronous replication ensures no data loss
- Cost: Approximately 2x single-AZ pricing

RDS Architecture Example

```
Highly Available Database Setup:

Primary DB: us-east-1a (handles all writes and reads)
Standby DB: us-east-1b (receives synchronous replication)

If us-east-1a fails:
1. RDS detects failure (health checks)
2. Automatically promotes standby in us-east-1b
3. Updates DNS endpoint to point to new primary
4. Application reconnects automatically

Result: ~60-120 second downtime, zero data loss
```

Aurora

Aurora is AWS's proprietary database, compatible with MySQL and PostgreSQL, offering up to 5x performance improvement.

- Storage: Auto-scales from 10 GB to 128 TB
- Replicas: Up to 15 read replicas with minimal lag
- Durability: 6 copies across 3 AZs
- Serverless Option: Aurora Serverless v2 for variable workloads

DynamoDB

DynamoDB is a fully managed NoSQL database service that provides fast and predictable performance with seamless scalability.

```
# DynamoDB Python Example (Boto3)
import boto3

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('Users')

# Put item
table.put_item(Item={
    'user_id': 'u123',
    'name': 'Alice',
    'email': 'alice@example.com'
})

# Get item
response = table.get_item(Key={'user_id': 'u123'})
print(response['Item'])
```

Feature	DynamoDB Capability
Performance	Single-digit millisecond latency at any scale
Scaling	Automatic, unlimited - no capacity planning needed
Consistency	Eventually consistent (default) or strongly consistent
Global Tables	Multi-region, active-active replication
Streams	Capture item-level changes for event-driven apps
DAX	In-memory cache for microsecond latency

Feature	DynamoDB Capability
Pricing	On-demand (pay per request) or provisioned (predictable)

Other AWS Database Services

Service	Type	Use Case
ElastiCache	Managed Redis/Memcached	Caching, session store, leaderboards
Redshift	Data warehouse	Analytics, BI reporting on petabytes
DocumentDB	MongoDB-compatible	Document store workloads
Keyspaces	Apache Cassandra-compatible	High-scale applications
Neptune	Graph database	Social networks, recommendation engines
Timestream	Time-series database	IoT, operational metrics
QLDB	Ledger database	Immutable transaction logs

AWS Networking

VPC (Virtual Private Cloud)

A VPC is a logically isolated section of AWS where you can launch resources in a virtual network you define.

```
VPC Architecture Example:

VPC (10.0.0.0/16)
├── Public Subnet (10.0.1.0/24) - us-east-1a
│   ├── Internet Gateway (for internet access)
│   ├── NAT Gateway (for private subnet outbound)
│   ├── Application Load Balancer
│   └── Bastion Host (jump box for admin access)
├── Private Subnet (10.0.2.0/24) - us-east-1a
│   └── Application Servers (no public IPs)
├── Private Subnet (10.0.3.0/24) - us-east-1b
```

```
│   └── Database Servers (highly isolated)
└── Route Tables, Security Groups, NACLs
```

VPC Key Components

Component	Purpose
Subnets	Divide VPC into network segments (public/private)
Route Tables	Define traffic routing rules between subnets
Internet Gateway	Connect VPC to the internet (one per VPC)
NAT Gateway	Allow private subnets to access internet (outbound only)
Security Groups	Instance-level firewall (stateful, allow rules only)
Network ACLs	Subnet-level firewall (stateless, allow/deny rules)
VPC Peering	Connect two VPCs privately
Transit Gateway	Hub for connecting multiple VPCs and on-premises

Route 53 (DNS Service)

Route 53 is AWS's scalable DNS and domain registration service.

- Domain Registration: Register and manage domain names
- DNS Routing: Translate domain names to IP addresses
- Health Checks: Monitor endpoint health and route around failures
- Routing Policies: Simple, weighted, latency-based, geo-location, failover

CloudFront (CDN)

CloudFront is a content delivery network that caches content at edge locations worldwide.

- Edge Locations: 400+ points of presence globally
- Origin Types: S3, EC2, ELB, custom HTTP servers
- Caching: Reduces latency and origin load
- Security: Integrates with AWS WAF and Shield for DDoS protection

- Pricing: Pay for data transfer and requests

Load Balancers

Type	Layer	Use Case	Features
ALB (Application)	Layer 7 (HTTP/HTTPS)	Web applications, microservices	Path-based routing, host-based routing, WebSocket
NLB (Network)	Layer 4 (TCP/UDP)	High performance, static IP	Millions of requests/sec, preserve client IP
GWLB (Gateway)	Layer 3/4	VPC endpoints, security appliances	Transparent inline load balancing
CLB (Classic)	Layer 4/7	Legacy applications	Deprecated, use ALB/NLB instead

AWS Security Services

IAM (Identity and Access Management)

IAM controls who can access what resources in your AWS account. It is the foundation of AWS security.

- Users: Individual people who need access (with passwords or access keys)
- Groups: Collections of users with similar permissions
- Roles: Temporary permissions that can be assumed by users, services, or applications
- Policies: JSON documents defining permissions (what actions on what resources)

```
# IAM Policy Example - Read-Only S3 Access
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
      "s3:GetObject",
      "s3:ListBucket"
      ],
```

```
      "Resource": [
        "arn:aws:s3:::my-bucket",
        "arn:aws:s3:::my-bucket/*"
      ]
    }
  ]
}
```

IAM Best Practices

0. Enable MFA for all users, especially root account
1. Use IAM roles instead of access keys for applications
2. Follow principle of least privilege - give minimum permissions needed
3. Rotate credentials regularly (access keys every 90 days)
4. Use IAM Access Analyzer to identify unintended access
5. Never share IAM users - one user per person

KMS (Key Management Service)

KMS makes it easy to create and manage cryptographic keys for data encryption.

- Customer Managed Keys (CMK): Full control over key lifecycle
- AWS Managed Keys: Automatically rotated every year
- Integration: Works with S3, EBS, RDS, Lambda, Secrets Manager, and more
- Pricing: $1/month per CMK + $0.03 per 10,000 API requests

Other Security Services

Service	Purpose
CloudTrail	Log every API call made in your AWS account (audit trail)
Config	Track resource configuration changes and compliance
GuardDuty	AI-powered threat detection (analyzes CloudTrail, VPC Flow Logs, DNS)
Security Hub	Centralized security findings dashboard across

Service	Purpose
	AWS services
WAF	Web Application Firewall - protect against SQL injection, XSS
Shield	DDoS protection (Standard: free, Advanced: managed response)
Secrets Manager	Store and rotate secrets (DB passwords, API keys)
Certificate Manager	Free SSL/TLS certificates for AWS resources

TIP: AWS Security Golden Rule

Enable CloudTrail in ALL regions, in ALL accounts, from day one. CloudTrail is your audit trail - without it, you cannot investigate security incidents or meet compliance requirements. It is free for the first copy of management events.

Part 3: Microsoft Azure

Azure is the second-largest cloud provider (~23% market share), deeply integrated with the Microsoft ecosystem (Windows Server, Active Directory, Office 365, Dynamics). It is the dominant choice for enterprises already using Microsoft technologies.

Azure Compute and Storage

Azure Virtual Machines

```
# Create a VM via Azure CLI
az vm create \
  --resource-group myResourceGroup \
  --name myVM \
  --image Ubuntu2204 \
  --size Standard_B2s \
  --admin-username azureuser \
  --generate-ssh-keys
```

VM Series	Optimized For	Equivalent AWS
B-series	Burstable, dev/test	T3
D-series	General purpose	M6i
F-series	Compute-intensive	C6i
E-series	Memory-intensive	R6i
N-series	GPU (ML, rendering)	P4/P5
L-series	Storage-intensive	I3

Azure App Service (PaaS)

Deploy web apps without managing infrastructure.

```
az webapp create --name my-web-app \
  --resource-group myRG \
  --plan myPlan \
  --runtime "PYTHON:3.11"
```

```
az webapp deployment source config-zip \
  --name my-web-app \
  --resource-group myRG \
  --src app.zip
```

Azure Functions (Serverless)

```
import azure.functions as func

def main(req: func.HttpRequest) -> func.HttpResponse:
    name = req.params.get('name', 'World')
    return func.HttpResponse(f"Hello, {name}!")
```

Azure Storage Services

Service	Type	Equivalent AWS
Blob Storage	Object storage	S3
Managed Disks	Block storage	EBS
Azure Files	SMB/NFS file share	EFS
Data Lake Storage Gen2	Big data analytics	S3 + Lake Formation
Archive Storage	Long-term archive	S3 Glacier

Blob Storage Tiers

Tier	Access Pattern	Cost Structure
Hot	Frequent access	Highest storage cost, lowest access cost
Cool	Infrequent (30+ days)	Lower storage, higher access cost
Cold	Rarely accessed (90+ days)	Even lower storage cost
Archive	Compliance/long-term (180+ days)	Cheapest storage, hours to retrieve

Azure Databases and Networking

Azure Database Services

Service	Type	Equivalent AWS
Azure SQL Database	Managed SQL Server	RDS for SQL Server
Azure SQL Managed Instance	Full SQL Server PaaS	—
Azure Database for PostgreSQL/MySQL	Managed open-source RDBMS	RDS
Cosmos DB	Multi-model NoSQL (global)	DynamoDB + DocumentDB
Azure Cache for Redis	Managed Redis	ElastiCache
Azure Synapse Analytics	Data warehouse + analytics	Redshift + Athena

Cosmos DB — Multi-Model APIs

Cosmos DB supports 5 different APIs, making it extremely versatile:

- NoSQL (Core/SQL): JSON documents with SQL-like queries
- MongoDB: Wire-compatible with MongoDB (drop-in replacement)
- Cassandra: Wide-column store for high-scale applications
- Gremlin: Graph database for relationships and connections
- Table: Key-value store (Azure Table Storage compatible)

TIP: Cosmos DB Superpower

Cosmos DB guarantees <10ms reads/writes globally with 99.999% availability. It is the only database that offers global distribution with single-digit millisecond latency at any scale.

Azure Networking

Service	Purpose	Equivalent AWS
Virtual Network (VNet)	Isolated network	VPC
NSG (Network Security Group)	Firewall rules	Security Groups
Azure Load Balancer	L4 load balancing	NLB

Service	Purpose	Equivalent AWS
Application Gateway	L7 load balancing + WAF	ALB + WAF
Azure Front Door	Global CDN + WAF + routing	CloudFront + Route 53
Azure DNS	DNS hosting	Route 53
ExpressRoute	Dedicated connection to Azure	Direct Connect
VPN Gateway	Site-to-site / Point-to-site VPN	AWS VPN

Azure Identity and Security

Azure Active Directory (Azure AD / Entra ID)

Azure AD is Microsoft's cloud-based identity and access management service - the heart of Azure security.

Feature	Description
SSO	Single Sign-On for 3,000+ SaaS apps
MFA	Multi-factor authentication
Conditional Access	Allow/block based on user, device, location, risk
RBAC	Role-Based Access Control on Azure resources
PIM	Privileged Identity Management - JIT admin access
B2C	Customer-facing identity (social logins)

Azure Security Services

Service	Purpose
Microsoft Defender for Cloud	CSPM + CWPP (posture + workload protection)
Azure Key Vault	Secrets, keys, and certificate management
Microsoft Sentinel	Cloud-native SIEM + SOAR
Azure DDoS Protection	Layer 3/4 DDoS mitigation

Service	Purpose
Azure Policy	Enforce governance rules across subscriptions

KQL Query Example (Log Analytics)

```
// Find failed sign-in attempts in the last 24 hours SigninLogs
| where TimeGenerated > ago(24h)
| where ResultType != "0"
| summarize FailedAttempts = count() by UserPrincipalName, IPAddress
| where FailedAttempts > 5
| order by FailedAttempts desc
```

Azure DevOps and Monitoring

Service	Purpose
Azure DevOps	Repos, Pipelines, Boards, Artifacts (full CI/CD platform)
Azure Monitor	Metrics, logs, alerts, dashboards
Application Insights	APM (Application Performance Monitoring)
Log Analytics	Centralized log querying (KQL)
Azure Automation	Runbooks for automated tasks
ARM Templates	Infrastructure as Code (JSON)
Bicep	Simplified IaC language (compiles to ARM)

Azure Well-Architected Framework

Pillar	Focus
Reliability	Availability zones, geo-redundancy, disaster recovery
Security	Identity, encryption, network protection
Cost Optimization	Right-sizing, reserved instances, auto-shutdown

Pillar	Focus
Operational Excellence	DevOps practices, monitoring, automation
Performance Efficiency	Scaling, caching, choosing the right services

Part 4: Google Cloud Platform (GCP)

GCP is the third-largest cloud provider (~11% market share), known for its strengths in data analytics, machine learning, Kubernetes (which Google invented), and global networking infrastructure. It powers Google Search, YouTube, and Gmail.

GCP Compute and Storage

Compute Engine (VMs)

```
# Create a VM
gcloud compute instances create my-instance \
  --zone=us-central1-a \
  --machine-type=e2-medium \
  --image-family=debian-12 \
  --image-project=debian-cloud \
  --boot-disk-size=50GB

# SSH into the VM
gcloud compute ssh my-instance --zone=us-central1-a
```

Machine Family	Use Case	Equivalent AWS
E2	General purpose, cost-optimized	T3
N2/N2D	Balanced performance	M6i
C2/C2D	Compute-intensive	C6i
M2/M3	Memory-intensive	R6i
A2/A3	GPU-accelerated (ML)	P4/P5
T2A	ARM-based (Tau)	Graviton

Google Kubernetes Engine (GKE)

GKE is the most advanced managed Kubernetes service - built by the team that created Kubernetes.

```
# Create a GKE cluster
```

```
gcloud container clusters create my-cluster \
  --zone us-central1-a \
  --num-nodes 3 \
  --machine-type e2-standard-4 \
  --enable-autoscaling --min-nodes=1 --max-nodes=10
```

- Autopilot Mode: Fully managed Kubernetes cluster - no node management
- Standard Mode: More control over cluster configuration
- Features: Built-in Istio, binary authorization, workload identity

Cloud Run (Serverless Containers)

Run containerized apps serverlessly - deploy any Docker container, scale to zero.

```
gcloud run deploy my-service \
  --image gcr.io/my-project/my-app:latest \
  --platform managed \
  --region us-central1 \
  --allow-unauthenticated
```

Cloud Storage

Storage Class	Access	Cost	Best For
Standard	Frequent	Highest	Active data, websites
Nearline	Once per month	Lower	Backups, media
Coldline	Once per quarter	Even lower	Disaster recovery
Archive	Once per year	Cheapest	Long-term compliance

GCP Data Analytics and ML

BigQuery — Serverless Data Warehouse

BigQuery is GCP's crown jewel for analytics - serverless, highly scalable, and incredibly fast.

```
-- Query 1 TB of data in seconds (serverless, pay per query)
SELECT
  state,
  COUNT(*) as num_trips,
```

```
  AVG(trip_miles) as avg_distance
FROM `bigquery-public-data.chicago_taxi_trips.taxi_trips`
WHERE trip_start_timestamp >= '2024-01-01'
GROUP BY state
ORDER BY num_trips DESC
LIMIT 10;
```

- Serverless: No infrastructure to manage, automatic scaling
- Performance: Process petabytes in seconds
- Pricing: $5 per TB scanned (or flat-rate for high usage)
- Built-in ML: CREATE MODEL directly in SQL

Cloud Spanner — The Unique Service

Cloud Spanner is the only database that offers global distribution with strong consistency AND horizontal scaling.

- Traditional databases force you to choose between consistency and scalability
- Spanner gives you both - Google's secret sauce
- Used by: Google Play, Gmail, Google Photos
- Best For: Financial systems, inventory management, global applications

GCP AI/ML Services

Service	Purpose
Vertex AI	End-to-end ML platform (training, deployment, monitoring)
AutoML	Train custom models without ML expertise
Cloud Vision AI	Image analysis, OCR
Cloud Natural Language	Text analysis, sentiment, entity extraction
Cloud Speech-to-Text	Audio transcription
Cloud Translation	100+ language translation
Gemini AI	Google's multimodal foundation model
TPUs	Custom hardware for ML training

TIP: TPU vs GPU

Google's TPUs are custom-designed for TensorFlow and JAX. They deliver better performance-per-dollar for large-scale ML training compared to NVIDIA GPUs. Use TPUs for training, GPUs for inference flexibility.

GCP Networking Advantage

GCP's network is fundamentally different from AWS and Azure:

- Global VPC: Spans all regions automatically (AWS/Azure VPCs are regional)
- Premium Tier: Uses Google's private fiber network (same as Google Search)
- Lower Latency: Better global performance for distributed applications
- Simplified Architecture: No need for complex VPC peering across regions

Part 5: Containers and Docker

Containers are lightweight, portable units that package an application with all its dependencies. Docker is the industry-standard tool for building, running, and distributing containers.

Docker Fundamentals

Containers vs Virtual Machines

Feature	Virtual Machine	Container
Size	GBs (full OS)	MBs (just app + deps)
Startup Time	Minutes	Seconds
Isolation	Full (separate kernel)	Process-level (shared kernel)
Performance	Near-native with overhead	Near-native, minimal overhead
Portability	Hypervisor-dependent	Runs anywhere Docker runs
Density	~10-20 per server	~100s per server

Image vs Container

- Image: A read-only template (blueprint) containing app code, runtime, libraries, and configuration
- Container: A running instance of an image (like an object from a class in OOP)
- Registry: Storage and distribution for images (Docker Hub, ECR, ACR, GCR)

Essential Docker Commands

```
# Pull an image from Docker Hub
docker pull nginx:latest

# Run a container
docker run -d --name my-web -p 8080:80 nginx:latest
# -d = detached (background)
# -p 8080:80 = map host port 8080 to container port 80

# List running
containers docker ps
```

```
# List all containers (including stopped)
docker ps -a

# View container logs
docker logs my-web

# Execute a command inside a running container
docker exec -it my-web /bin/bash

# Stop and remove
docker stop my-web
docker rm my-web

# Remove an image
docker rmi nginx:latest

# Clean up unused resources
docker system prune -a
```

Dockerfile and Multi-Stage Builds

Writing a Dockerfile

```
# Use official Python image as base
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Copy dependency file first (layer caching optimization)
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

# Expose port
```

```
EXPOSE 8000

# Set environment variable
ENV FLASK_ENV=production

# Health check
HEALTHCHECK --interval=30s --timeout=5s \
  CMD curl -f http://localhost:8000/health || exit
  1

# Run the application
CMD ["python", "app.py"]


# Build the image
docker build -t my-app:1.0 .

# Run it
docker run -d -p 8000:8000 my-app:1.0
```

Multi-Stage Builds

Multi-stage builds create smaller, more secure production images by separating build and runtime environments.

```
# Stage 1: Build
FROM node:20 AS builder
WORKDIR /app
COPY package*.json
./ RUN npm ci
COPY . .
RUN npm run build

# Stage 2: Production (tiny image)
FROM nginx:alpine
COPY --from=builder /app/dist
/usr/share/nginx/html EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]

# Result: ~25 MB image instead of ~1 GB with build tools
```

Docker Compose and Networking

Docker Compose

Docker Compose defines and runs multi-container applications.

```
# docker-compose.yml
version: '3.8'

services:
  web:
    build: ./web
    ports:
      - "3000:3000"
    environment:
      - DATABASE_URL=postgres://user:pass@db:5432/mydb
    depends_on:
      - db

  db:
    image: postgres:16
    volumes:
      - pgdata:/var/lib/postgresql/
    data environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: mydb

volumes:
  pgdata:


# Start all services
docker compose up -d

# View logs
docker compose logs -f

# Stop and clean up
docker compose down -v
```

Docker Networking

Network Type	Description	Use Case
bridge (default)	Isolated network on a single host	Dev environments, standalone containers
host	Container shares host's network stack	Performance-critical apps
overlay	Multi-host networking (Swarm/K8s)	Production distributed systems
none	No networking	Security-isolated containers

Docker Security Best Practices

6. Use minimal base images (alpine, distroless, slim variants)
7. Don't run as root - use USER 1000 in Dockerfile
8. Scan images for CVEs (docker scout, Trivy, Snyk)
9. Use multi-stage builds to keep build tools out of production
10. Pin image versions - FROM python:3.11.7-slim not python:latest
11. No secrets in images - use Docker secrets or environment variables
12. Use read-only filesystem where possible
13. Limit resources - docker run --memory=512m --cpus=1.0
14. Never use --privileged in production

Part 6: Kubernetes

Kubernetes is the industry-standard platform for container orchestration - automating the deployment, scaling, and management of containerized applications. Originally developed by Google (based on their internal Borg system), it is now maintained by the CNCF.

Kubernetes Architecture

A Kubernetes cluster consists of a Control Plane (master) and Worker Nodes.

Component	Role	Description
API Server	Control Plane	Gateway to the cluster; processes kubectl commands
etcd	Control Plane	Distributed key-value store; holds ALL cluster state
Scheduler	Control Plane	Decides which node to place new pods on
Controller Manager	Control Plane	Ensures desired state matches actual state
kubelet	Worker Node	Agent on each node; manages pod lifecycle
kube-proxy	Worker Node	Handles networking rules and service routing

Kubernetes Objects

Pod — The Smallest Unit

```
apiVersion: v1
kind: Pod
metadata:
name: my-app
labels:
  app: web
spec:
 containers:
```

```
  - name: web
    image: nginx:1.25
    ports:
    - containerPort: 80
    resources:
    requests:
        memory: "128Mi"
      cpu: "250m"
      limits:
        memory: "256Mi"
        cpu: "500m"
```

A Pod wraps one or more containers that share networking and storage.

Deployment — Manages Pod Replicas

```
apiVersion:
apps/v1 kind:
Deployment
metadata:
  name: web-deployment
spec:
  replicas: 3
  selector:
  matchLabels:
      app: web
  strategy:
    type: RollingUpdate
    rollingUpdate:
    maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: web
    spec:
    containers:
      - name: web
        image: my-app:2.0
        ports:
        - containerPort: 8080
```

Service — Network Access to Pods

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: LoadBalancer
  selector:
    app: web
  ports:
  - port: 80
    targetPort: 8080
```

Service Type	Access	Use Case
ClusterIP	Internal only (default)	Service-to-service communication
NodePort	External via node IP:port	Development, testing
LoadBalancer	External via cloud LB	Production (AWS ALB, GCP LB)
ExternalName	DNS alias to external service	Access external APIs

ConfigMap & Secret

```yaml
# ConfigMap — non-sensitive config
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  LOG_LEVEL: "info"
  API_URL: "https://api.example.com"

---
# Secret — sensitive data (base64 encoded)
apiVersion: v1
kind: Secret
metadata:
  name:  db-secret
```

```
type: Opaque
data:
  DB_PASSWORD: cGFzc3dvcmQxMjM=  # base64 encoded
```

Ingress — HTTP Routing

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
        service:
          name: api-service
          port:
            number: 8080
      - path: /
        pathType: Prefix
        backend:
        service:
          name: web-service
          port:
            number: 80
```

Helm and kubectl Commands

Helm — Package Manager for Kubernetes

```
# Add a Helm repo
helm repo add bitnami https://charts.bitnami.com/bitnami

# Install PostgreSQL
```

```
helm install my-postgres bitnami/postgresql \
  --set auth.postgresPassword=mypassword \
  --set primary.persistence.size=50Gi

# List releases
helm list

# Upgrade
helm upgrade my-postgres bitnami/postgresql \
  --set auth.postgresPassword=newpass

# Uninstall
helm uninstall my-postgres
```

Essential kubectl Commands

```
# Cluster info
kubectl cluster-info
kubectl get nodes

# Working with pods
kubectl get pods -A                # All namespaces
kubectl describe pod my-app        # Detailed info
kubectl logs my-app -f             # Stream logs
kubectl exec -it my-app -- /bin/bash  # Shell into
pod

# Deployments
kubectl apply -f deployment.yaml      # Create/update resources
kubectl rollout status deployment/web  # Watch rollout
kubectl rollout undo deployment/web   # Rollback
kubectl scale deployment web --replicas=5

# Debugging
kubectl get events --sort-by='.lastTimestamp'
kubectl top pods                   # Resource usage
kubectl describe node node-1       # Node details
```

Horizontal Pod Autoscaler (HPA)

```
apiVersion: autoscaling/v2
```

```
kind: HorizontalPodAutoscaler
metadata:
  name: web-hpa
spec:
scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-deployment
  minReplicas: 2
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

TIP: Kubernetes Learning Path

Start with kubectl basics, then learn Deployments and Services. Once comfortable, explore ConfigMaps, Secrets, and Ingress. Finally, learn Helm for package management and HPA for auto-scaling. The CKA (Certified Kubernetes Administrator) certification is highly valued in the industry.

Part 7: Serverless Computing

Serverless computing lets you run code without provisioning or managing servers. The cloud provider handles infrastructure, auto-scaling, and high availability. You pay only for the compute time your code actually consumes - down to the millisecond.

Function as a Service (FaaS)

What Serverless Really Means

```
Traditional:              Serverless:
You manage:                You manage:
- Servers                 - Code (functions)
- OS                      - Configuration
- Runtime
- Scaling
- Patching
- Availability

                  Provider manages everything else
```

Serverless does NOT mean 'no servers.' Servers exist - you just don't see, manage, or pay for idle ones.

FaaS Platforms Comparison

Platform	Provider	Max Duration	Best For
AWS Lambda	AWS	15 minutes	Most use cases, extensive integrations
Azure Functions	Microsoft	10 min (Consumption)	Microsoft ecosystem integration
Google Cloud Functions	Google	60 min (2nd gen)	GCP-native applications
Cloudflare Workers	Cloudflare	15 minutes (paid)	Edge computing, low latency
Vercel Functions	Vercel	5 minutes (Pro)	Frontend-focused applications

Event-Driven Architecture

Serverless functions are triggered by events, not direct HTTP calls (though HTTP is one trigger type).

Event Source	Example	Use Case
HTTP Request	API Gateway → Lambda	REST APIs, webhooks
Object Storage	S3 upload → Lambda	Image resizing, video transcoding
Message Queue	SQS message → Lambda	Order processing, async tasks
Database Change	DynamoDB Stream → Lambda	Real-time data sync
Schedule	CloudWatch Cron → Lambda	Nightly reports, cleanup jobs
IoT	IoT sensor → Lambda	Data ingestion, alerting

Image Processing Pipeline

```
User uploads photo to S3
    │
    ↓
S3 Event triggers Lambda
    │
    ↓
Lambda resizes image (thumbnail + full size)
    │
    ├──→ Saves thumbnails back to S3
    ├──→ Writes metadata to DynamoDB
    └──→ Sends SNS notification to user
```

Cold Starts and Best Practices

Understanding Cold Starts

When a function hasn't been invoked recently, the cloud provider must:

15. Allocate a new container
16. Download the function code
17. Initialize the runtime
18. Run initialization code (imports, DB connections)

This delay is called a 'cold start.'

Runtime	Typical Cold Start
Python	200-500 ms
Node.js	150-400 ms
Go	50-150 ms
Java	1,000-5,000 ms (JVM startup)
.NET	500-2,000 ms
Rust (custom)	10-50 ms

Reducing Cold Starts

Strategy	How
Provisioned Concurrency	Pre-warm N instances (AWS Lambda)
Minimize package size	Remove unused dependencies
Use lighter runtimes	Go/Rust instead of Java
Keep functions warm	Periodic ping (CloudWatch + Lambda)
Connection pooling	Initialize DB connections outside handler
Use SnapStart	AWS Lambda SnapStart for Java

Serverless Frameworks

```
# AWS SAM (Serverless Application Model)
AWSTemplateFormatVersion: '2010-09-09'
Transform: AWS::Serverless-2016-10-31

Resources:
  HelloFunction:
    Type: AWS::Serverless::Function
    Properties:
      Handler: app.handler
      Runtime: python3.11
      Events:
```

```
ApiEvent:
  Type: Api
  Properties:
  Path: /hello
  Method: get
```

When NOT to Use Serverless

Scenario	Why Not	Better Alternative
Long-running processes (>15 min)	Lambda timeout limit	EC2, ECS, Cloud Run
High-throughput, steady traffic	Cost may exceed reserved instances	Containers, VMs
Requires persistent connections	WebSockets have limitations	ECS, Cloud Run
GPU workloads	Lambda doesn't support GPUs	EC2 + GPU instances
Extremely low latency	Cold starts unacceptable	Containers with warm pools

Part 8: Infrastructure as Code

Infrastructure as Code transforms infrastructure provisioning from manual, error-prone clicking into version-controlled, repeatable, automated code. IaC is the foundation of DevOps and cloud engineering.

Terraform Deep Dive

Why IaC?

Aspect	Manual (ClickOps)	Infrastructure as Code
Consistency	Drift between environments	Identical Dev/Staging/Prod
Speed	Hours of clicking	Minutes of terraform apply
Version Control	No history	Git tracks every change
Review	No peer review	Pull requests for infrastructure
Disaster Recovery	'Rebuild from memory'	terraform apply = rebuilt in minutes
Documentation	Outdated wikis	Code IS the documentation

Terraform Core Concepts

```
terraform init    → Download provider plugins
terraform plan    → Preview what will change
terraform apply   → Create/update infrastructure
terraform destroy → Delete everything
```

Complete Terraform Example

```
# provider.tf
terraform {
required_providers {
  aws = {
   source =
   "hashicorp/aws" version
   = "~> 5.0"
  }
```

```
  }
  backend "s3" {
    bucket = "my-terraform-state"
    key    = "prod/terraform.tfstate"
    region = "us-east-1"
  }
}

provider "aws" {
  region = "us-east-1"
}

# vpc.tf
resource "aws_vpc" "main" {
  cidr_block          =
  "10.0.0.0/16"
  enable_dns_hostnames =
  true tags = {
    Name = "production-vpc"
  }
}

resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.0.1.0/24"
  availability_zone       = "us-east-1a"
  map_public_ip_on_launch = true
  tags = { Name = "public-subnet" }
}

# outputs.tf
output "vpc_id" {
  value = aws_vpc.main.id
}
```

Terraform State Management

terraform.tfstate is the source of truth for what infrastructure exists.

WARNING: Critical Warning

Terraform state contains sensitive data (resource IDs, sometimes secrets). Never commit state to Git! Use remote backend (S3 + DynamoDB for locking) for team collaboration.

Terraform Modules

```
# Reusable module for creating a web server
module "web_server" {
  source        = "./modules/ec2-web"
  instance_type = "t3.medium"
  environment   = "production"
  vpc_id        =
  module.network.vpc_id
  subnet_id     = module.network.public_subnet_id
}

module  "staging_web"  {
  source        = "./modules/ec2-
  web" instance_type = "t3.small"
  environment   = "staging"
  vpc_id        = module.staging_network.vpc_id
  subnet_id     = module.staging_network.public_subnet_id
}
```

CloudFormation and Pulumi

AWS CloudFormation

AWS-native IaC using YAML/JSON templates.

```
AWSTemplateFormatVersion: '2010-09-09'
Description: Web server stack

Parameters:
  InstanceType:
    Type: String
    Default: t3.medium

Resources:
  WebServer:
    Type: AWS::EC2::Instance
    Properties:
      InstanceType: !Ref InstanceType
      ImageId: ami-0abcdef1234567890
      Tags:
        - Key: Name
```

```
      Value: WebServer

Outputs:
  ServerIP:
    Value: !GetAtt WebServer.PublicIp
```

Pulumi (IaC with Programming Languages)

```
# Pulumi — Python
import pulumi
import pulumi_aws as aws

vpc = aws.ec2.Vpc("main-vpc", cidr_block="10.0.0.0/16")

subnet = aws.ec2.Subnet("public-subnet",
    vpc_id=vpc.id,
    cidr_block="10.0.1.0/24",
    map_public_ip_on_launch=True
)

instance = aws.ec2.Instance("web-server",
    instance_type="t3.medium",
    ami="ami-0abcdef1234567890",
    subnet_id=subnet.id,
    tags={"Name": "WebServer"}
)

pulumi.export("public_ip", instance.public_ip)
```

Terraform vs Ansible

Feature	Terraform	Ansible
Purpose	Provision infrastructure	Configure existing infrastructure
Approach	Declarative (desired state)	Procedural (tasks in order)
State	Maintains state file	Stateless
Agents	Agentless (API-based)	Agentless (SSH-based)

Feature	Terraform	Ansible
Best For	Creating cloud resources	Installing packages, configs

They are complementary: Terraform creates the servers, Ansible configures them.

IaC Best Practices

19. Version control everything - All IaC code in Git
20. Use modules/templates - DRY (Don't Repeat Yourself)
21. Remote state + locking - S3 + DynamoDB for Terraform
22. Environment separation - Separate state files per environment
23. Plan before apply - Always review terraform plan output
24. Use variables & secrets management - Never hardcode credentials
25. Implement CI/CD for IaC - PR → Plan → Review → Merge → Apply
26. Policy as Code - Use OPA, Sentinel, or Checkov to enforce rules
27. Tagging strategy - Tag all resources (Name, Environment, Team)
28. Drift detection - Regularly compare actual vs declared state

Part 9: CI/CD and DevOps

CI/CD (Continuous Integration / Continuous Delivery) is the automation backbone of modern software engineering. It ensures that every code change is automatically built, tested, and deployed - reducing deployment risk from 'nerve-wracking Friday deploys' to 'boring, routine events happening 50 times a day.'

CI/CD Pipelines and GitOps

The DevOps Lifecycle

```
Plan ←──────────────────────────────┐
                                    │
Code → Build → Test → Release → Deploy → Operate → Monitor
──────── CI ──────────── ─── CD ─────── ─── Ops ──────────
```

CALMS Framework

Pillar	Description
Culture	Break silos between Dev and Ops teams
Automation	Automate everything repeatable
Lean	Reduce waste, small batch sizes
Measurement	Track metrics (DORA, SLIs, SLOs)
Sharing	Share knowledge, tools, and responsibility

CI Pipeline Stages

```
1. Code Commit     → Developer pushes to Git
2. Build           → Compile code, install dependencies
3. Unit Tests      → Run fast, isolated tests
4. Static Analysis → Linting, code quality, SAST security scanning
5. Integration Tests → Test with databases, APIs
6. Artifact Build  → Create Docker image, binary, package
7. Artifact Push   → Store in registry (ECR, ACR, Artifact Registry)
```

GitHub Actions Example

```yaml
name: CI Pipeline
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          pip install pytest coverage flake8

      - name: Lint
        run: flake8 src/ --max-line-length=120

      - name: Run tests with coverage
        run: pytest tests/ --cov=src --cov-report=xml

      - name: Upload coverage
        uses: codecov/codecov-action@v3

  build:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Build Docker image
        run: docker build -t my-app:${{ github.sha }} .
```

```
- name: Push to ECR
  run: |
    aws ecr get-login-password | docker login --username AWS --password-stdin $ECR_REGISTRY
    docker push $ECR_REGISTRY/my-app:${{ github.sha }}
```

CI/CD Platforms Comparison

Platform	Best For	Configuration
GitHub Actions	GitHub-hosted repos	YAML
GitLab CI/CD	GitLab-hosted repos	YAML
Jenkins	Self-hosted, highly customizable	Groovy (Jenkinsfile)
CircleCI	Speed, Docker-first	YAML
Azure DevOps	Microsoft ecosystem	YAML
Google Cloud Build	GCP integration	YAML/JSON
Argo Workflows	Kubernetes-native CI	YAML

Deployment Strategies

Continuous Delivery vs Deployment

Term	Definition
Continuous Delivery	Code is always in a deployable state. Deployment requires manual approval.
Continuous Deployment	Every change that passes tests is automatically deployed to production.

Deployment Strategies Comparison

Strategy	Description	Risk	Rollback
Rolling Update	Replace instances gradually, one by one	Low	Slow (reverse the rolling update)

Strategy	Description	Risk	Rollback
Blue-Green	Run two identical environments. Switch traffic.	Minimal	Instant (switch back to Blue)
Canary	Deploy to small % of traffic first, then increase	Lowest	Instant (route away from canary)
A/B Testing	Route different user segments to different versions	Low	Per-segment
Recreate	Kill all old instances, start new ones	High (downtime!)	Redeploy old version

Blue-Green Deployment

```
Before:
Load Balancer → Blue (v1.0) [ACTIVE]
                Green (v1.0) [IDLE]

Deploy v2.0 to Green:
Load Balancer → Blue (v1.0) [ACTIVE]
                Green (v2.0) [TESTING]

Switch traffic:
Load Balancer → Green (v2.0) [ACTIVE]
                Blue (v1.0) [STANDBY — instant rollback if needed]
```

Canary Deployment

```
Step 1: 5% traffic → v2.0, 95% → v1.0   (monitor errors, latency)
Step 2: 25% traffic → v2.0, 75% → v1.0  (if metrics OK, continue)
Step 3: 50% → v2.0, 50% → v1.0
Step 4: 100% → v2.0                     (full rollout)

If errors spike at any step → automatic rollback to v1.0
```

Observability and Monitoring

Three Pillars of Observability

Pillar	What It Captures	Tools
Metrics	Numerical measurements over time (CPU, latency, errors)	Prometheus, Grafana, CloudWatch, Datadog
Logs	Text records of events	ELK Stack, Loki, CloudWatch Logs, Splunk
Traces	Request flow across distributed services	Jaeger, Zipkin, AWS X-Ray, OpenTelemetry

DORA Metrics

DORA (DevOps Research & Assessment) metrics measure software delivery performance:

Metric	Elite	High	Medium	Low
Deployment Frequency	Multiple/day	Weekly	Monthly	Yearly
Lead Time for Changes	< 1 hour	1 day-1 week	1-6 months	> 6 months
Mean Time to Restore	< 1 hour	< 1 day	1 day-1 week	> 6 months
Change Failure Rate	0-15%	16-30%	16-30%	> 30%

SLIs, SLOs, SLAs

Term	Definition	Example
SLI (Service Level Indicator)	A measured metric	99.8% of requests complete in < 200ms
SLO (Service Level Objective)	Target for the SLI	99.9% availability over 30 days
SLA (Service Level Agreement)	Legal contract with consequences	99.95% or we refund 10% of bill

GitOps

GitOps uses Git as the single source of truth for both application code AND infrastructure state.

- Declarative: All infrastructure and app config defined in Git
- Versioned: Git history = audit trail of all changes
- Automated: Changes in Git are automatically applied to the cluster
- Observable: System continuously reconciles actual state with Git state

Tool	Purpose
ArgoCD	Kubernetes GitOps operator (most popular)
Flux	Kubernetes GitOps toolkit (CNCF project)
Jenkins X	CI/CD + GitOps for Kubernetes

Part 10: Cloud Architecture Patterns

Cloud architecture is the art and science of designing systems that are reliable, scalable, cost-effective, and secure in the cloud.

Well-Architected Framework

All three major cloud providers publish a 'Well-Architected Framework' with similar pillars:

Pillar	Focus
Operational Excellence	Automate operations, learn from failures
Security	Protect data, manage identity, detect threats
Reliability	Recover from failures, scale to meet demand
Performance Efficiency	Use resources efficiently, adopt new tech
Cost Optimization	Eliminate waste, right-size, use pricing models
Sustainability	Minimize environmental impact

Microservices and 12-Factor App

The Twelve-Factor App

A methodology for building modern, cloud-native applications:

Factor	Principle	Implementation
1. Codebase	One codebase tracked in Git, many deploys	Single repo → dev/staging/prod
2. Dependencies	Explicitly declare and isolate	requirements.txt, package.json
3. Config	Store config in environment variables	DATABASE_URL=postgres://...
4. Backing Services	Treat databases, queues as attached resources	Swap PostgreSQL for Aurora
5. Build, Release, Run	Strictly separate build and run	CI builds → immutable artifacts

Factor	Principle	Implementation
	stages	→ deploy
6. Processes	Execute app as stateless processes	No local file storage; use S3/Redis
7. Port Binding	Export services via port binding	App binds to port 8080
8. Concurrency	Scale out via the process model	Horizontal scaling, not vertical
9. Disposability	Fast startup, graceful shutdown	Containers start in seconds
10. Dev/Prod Parity	Keep environments as similar as possible	Same Docker image in dev and prod
11. Logs	Treat logs as event streams	Log to stdout → collected by Fluentd
12. Admin Processes	Run admin tasks as one-off processes	Database migrations as separate containers

Monolith vs Microservices

Scenario	Monolith	Microservices
Small team (< 10)	Recommended	Overkill
Startup/MVP	Recommended	Too complex
Large organization	Difficult to scale teams	Recommended
Independent team scaling	Not possible	Each team owns their service

Cloud Design Patterns

Pattern	Description	Implementation
Circuit Breaker	Stop calling a failing service, fallback gracefully	Hystrix, Resilience4j, Istio
Retry with Backoff	Retry failed requests with exponential delay	SDK built-in, custom middleware
Bulkhead	Isolate failures to prevent	Separate thread pools per

Pattern	Description	Implementation
	cascade	service
Timeout	Don't wait forever for a response	Set timeouts on all HTTP calls
Fallback	Return cached/default data when service is down	Cache-aside pattern
Health Check	Expose endpoint for monitoring	/health returns 200 OK

High Availability and Disaster Recovery

HA Architecture

```
Region: us-east-1
├── AZ-a: [Web] [App] [DB Primary]
├── AZ-b: [Web] [App] [DB Standby]
└── AZ-c: [Web] [App]

Load Balancer distributes across AZs
DB: Multi-AZ with automatic failover
```

Disaster Recovery Strategies

Strategy	RTO	RPO	Cost
Backup & Restore	Hours	Hours	$
Pilot Light	Minutes-Hours	Minutes	$$
Warm Standby	Minutes	Seconds-Minutes	$$$
Multi-Site Active-Active	Near-zero	Near-zero	$$$$

- RTO (Recovery Time Objective): How fast must you recover?
- RPO (Recovery Point Objective): How much data loss is acceptable?

Cost Optimization (FinOps)

FinOps Lifecycle

1. Inform → Visualize spending (who, what, where)
2. Optimize → Right-size, reserved instances, spot
3. Operate → Budget alerts, tagging policies, chargeback

Cost Optimization Tactics

Tactic	Potential Savings
Right-size instances	20-40%
Reserved Instances / Savings Plans	30-72%
Spot/Preemptible instances	60-90%
Auto-scaling to zero (nights/weekends)	30-50%
Storage lifecycle policies	20-40%
Delete unused resources	Immediate
Optimize data transfer	10-30%

FinOps Tools

Tool	Provider
AWS Cost Explorer	AWS
Azure Cost Management	Azure
GCP Billing Reports	GCP
Kubecost	Kubernetes cost allocation
Infracost	Cost estimation for Terraform
CloudHealth / Spot.io	Multi-cloud cost management

Part 11: Interview Preparation

This section prepares you for cloud computing interviews with common questions, scenarios, and career guidance.

Common Interview Questions

Beginner Level

Q1: What is the difference between IaaS, PaaS, and SaaS?

These are the three main cloud service models:

IaaS (Infrastructure as a Service): Provides virtualized computing resources like servers, storage, and networking. You manage the OS and applications. Examples: AWS EC2, Azure VMs.

PaaS (Platform as a Service): Provides a platform for developing and deploying applications. You manage only your code and data. Examples: AWS Elastic Beanstalk, Google App Engine.

SaaS (Software as a Service): Provides fully functional applications over the internet. You just use the software. Examples: Gmail, Salesforce, Office 365.

Q2: Explain the Shared Responsibility Model.

The Shared Responsibility Model divides security responsibilities:

Provider Responsibility (Security OF the Cloud): Physical security, network infrastructure, hypervisor, host operating system.

Customer Responsibility (Security IN the Cloud): Data protection, access management, application security, network configuration, OS patches.

The division varies by service model: IaaS requires more customer responsibility, while SaaS shifts most responsibility to the provider.

Q3: What is an Availability Zone?

An Availability Zone (AZ) is an isolated data center within a cloud region. Each region has multiple AZs (typically 2-6) that are physically separate but connected by low-latency links.

Importance:
- High Availability: Deploy across multiple AZs for fault tolerance
- Disaster Recovery: If one AZ fails, others continue operating
- Low Latency: AZs in same region have minimal network delay

Best Practice: Always deploy production workloads across at least 2 AZs.

Intermediate Level

Q4: How does auto-scaling work?

Auto-scaling automatically adjusts the number of compute resources based on demand.

How It Works:
1. Define scaling policies based on metrics (CPU, memory, requests)
2. Set minimum and maximum instance limits
3. CloudWatch monitors metrics continuously
4. When threshold is crossed, auto-scaling adds or removes instances
5. Load balancer distributes traffic to new instances

Benefits: Cost optimization, performance, availability, no manual intervention.

Q5: Explain S3 storage classes.

S3 offers multiple storage classes optimized for different access patterns:

S3 Standard: Frequently accessed data. 99.99% availability.

S3 Intelligent-Tiering: Unknown or changing access patterns. Auto-optimizes.

S3 Standard-IA: Infrequently accessed data (monthly). Lower cost, retrieval

fee. S3 Glacier: Archives (quarterly access). Very low cost, minutes-hours

retrieval.

S3 Glacier Deep Archive: Long-term retention (yearly). Lowest cost, hours retrieval.

Q6: What is a VPC?

A VPC (Virtual Private Cloud) is a logically isolated section of the cloud where you launch resources in a virtual network you define.

Key Components:
- Subnets: Network segments (public/private)
- Route Tables: Traffic routing rules
- Internet Gateway: Connect VPC to internet
- Security Groups: Instance-level firewall

Benefits: Network isolation, security control, custom IP range, hybrid connectivity.

Advanced Level

Q7: Design a multi-region, highly available architecture.

Architecture for global high availability:

DNS Layer: Route 53 with latency-based routing and health checks

Application Layer (per region):
- CloudFront CDN at edge locations
- Application Load Balancer across 3 AZs
- Auto Scaling Group with health checks
- ECS/EKS for container orchestration

Data Layer:
- DynamoDB Global Tables (multi-region, active-active)
- RDS with cross-region read replicas
- S3 with cross-region replication

Disaster Recovery: Pilot light in secondary region, automated failover runbooks.

Q8: How would you secure a cloud environment?

Defense-in-depth security strategy:

Identity and Access:
- Enable MFA for all users
- Use IAM roles instead of access keys
- Implement least privilege access

Network Security:
- VPC with private subnets
- Security groups and NACLs

- WAF for web applications

Data Protection:
- Encryption at rest (KMS) and in transit (TLS)
- Secrets management

Monitoring:
- CloudTrail for API auditing
- GuardDuty for threat detection

Scenario-Based Questions

Scenario 1: Migrate a Legacy Application to Cloud

Question: A company has a 3-tier web application running on-premises. How would you migrate it to AWS?

Answer:

Assessment Phase:
1. Inventory existing infrastructure and dependencies
2. Identify performance requirements and constraints
3. Evaluate migration strategies (rehost, replatform, refactor)

Migration Strategy - Replatform:
1. Database: Migrate to RDS using AWS DMS
2. Application: Deploy on EC2 with Auto Scaling
3. Web Tier: Use Application Load Balancer
4. Storage: Move static assets to S3 with CloudFront
5. Caching: Add ElastiCache Redis

Post-Migration:
1. Optimize instance types based on CloudWatch metrics
2. Implement CI/CD pipeline
3. Set up monitoring and alerting

Scenario 2: Handle Black Friday Traffic

Question: An e-commerce site expects 10x traffic during Black Friday. How do you prepare?

Answer:

Preparation:
1. Load Testing: Simulate 10x traffic to identify bottlenecks
2. Auto Scaling: Configure aggressive scaling policies
3. Caching: Implement CloudFront CDN, ElastiCache
4. Database: Enable read replicas, consider DynamoDB

Architecture Adjustments:
1. Static Content: Serve from CloudFront/S3
2. API Layer: Use API Gateway with caching
3. Queue: Implement SQS for order processing
4. Database: Enable Aurora Auto Scaling

Monitoring: Real-time dashboards, alerts, on-call team.

Career Path and Certifications

Cloud Career Roles

Role	Responsibilities	Skills Needed
Cloud Engineer	Deploy and manage cloud infrastructure	IaC, networking, automation
Cloud Architect	Design cloud solutions and strategy	Architecture patterns, multiple clouds
DevOps Engineer	CI/CD, automation, monitoring	CI/CD tools, containers, scripting
Cloud Security Engineer	Secure cloud environments	Security tools, compliance, IAM
SRE	System reliability and performance	Monitoring, automation, troubleshooting

Top Cloud Certifications

Certification	Provider	Level
AWS Cloud Practitioner	AWS	Foundational

Certification	Provider	Level
AWS Solutions Architect Associate	AWS	Associate
AWS DevOps Engineer Professional	AWS	Professional
Azure Fundamentals (AZ-900)	Microsoft	Foundational
Azure Administrator (AZ-104)	Microsoft	Associate
GCP Cloud Engineer	Google	Associate
CKA (Kubernetes Administrator)	CNCF	Intermediate
Terraform Associate	HashiCorp	Associate

90-Day Cloud Learning Plan

```
Week 1-2: Fundamentals
- Complete Cloud Practitioner course
- Set up free tier account
- Launch first EC2 instance

Week 3-4: Core Services
- Learn S3, RDS, VPC in depth
- Build a simple 3-tier application
- Practice IAM and security

Week 5-8: Associate Level
- Study for Solutions Architect Associate
- Complete hands-on labs
- Take practice exams

Week 9-12: Advanced Topics
- Learn containers and Kubernetes
- Study serverless architectures
- Build a portfolio project

Ongoing: Stay Current
- Follow cloud blogs and news
```

- Attend webinars
- Contribute to community

Hands-On Labs and Projects

Beginner Projects

- Project 1: Deploy a static website on S3 with CloudFront
- Project 2: Create a VPC with public and private subnets
- Project 3: Set up an EC2 instance with automated backups
- Project 4: Build a serverless API with Lambda and API Gateway

Intermediate Projects

- Project 5: Deploy a 3-tier application with auto-scaling
- Project 6: Containerize an application with Docker
- Project 7: Deploy to Kubernetes with CI/CD pipeline
- Project 8: Implement Infrastructure as Code with Terraform

Advanced Projects

- Project 9: Build a multi-region disaster recovery solution
- Project 10: Implement GitOps with ArgoCD
- Project 11: Create a data pipeline with streaming
- Project 12: Design and implement a microservices architecture

TIP: Final Advice

Cloud computing is a journey, not a destination. Start with the fundamentals, practice with hands-on labs, and build real projects. Certifications validate your knowledge but hands-on experience is what employers value most. Stay curious, keep learning, and enjoy building in the cloud!

Cloud Computing

Complete A to Z Guide

Created & Authored by

Mohammad Faiz

Copyright (c) 2026 Mohammad Faiz. All Rights Reserved.

No Permission to Use: No person or entity is permitted to use, copy, modify, merge, publish, distribute, sublicense, or sell any part of this content.

Personal Use Only: Only the original author Mohammad Faiz has the authority to use this content in any project or application.